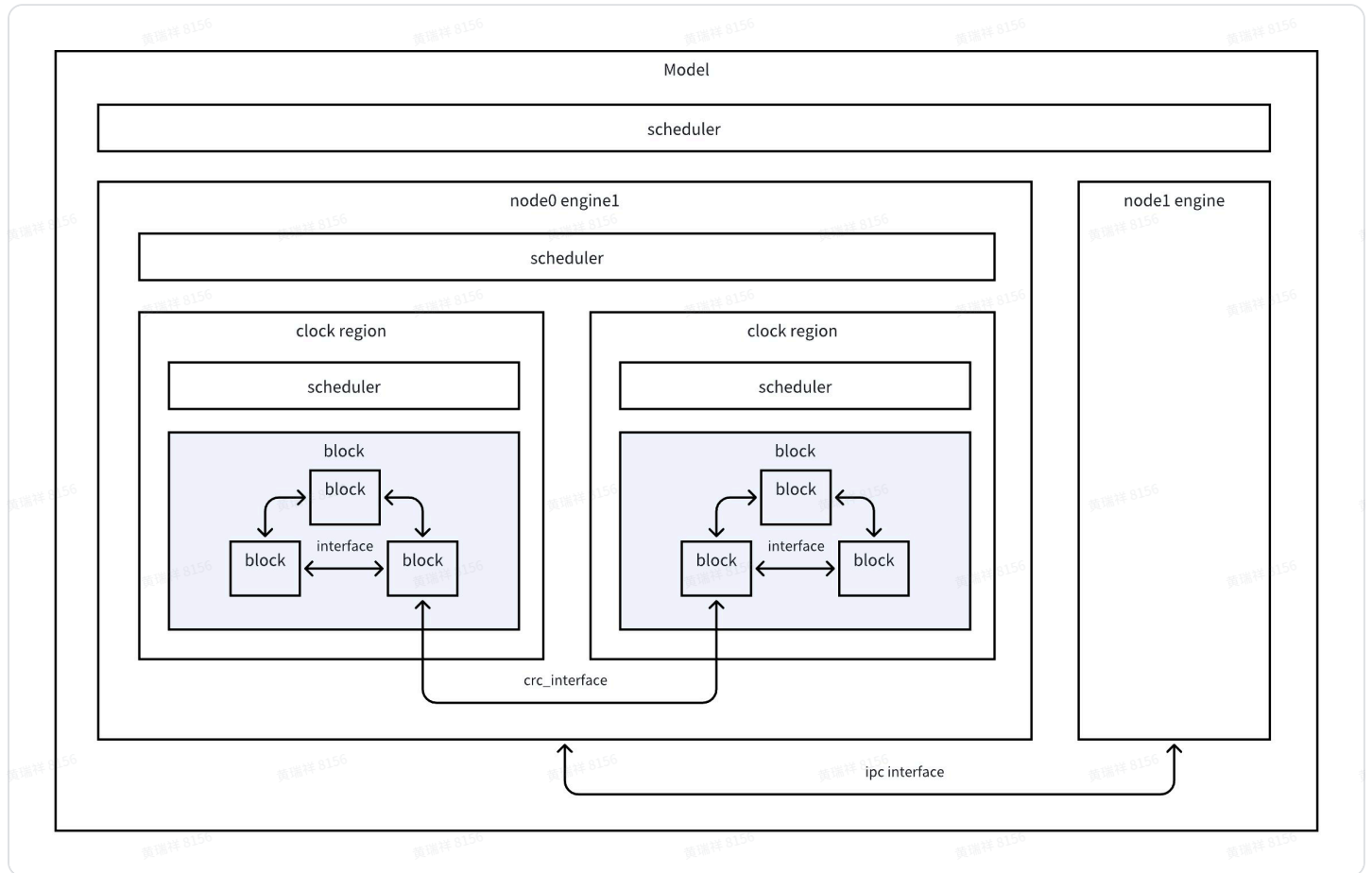


SiPU ESL Model Framework

1. Hierarchy



代码块

```

1  class EslEngine : public EslComponent {
2  public:
3      template <typename T, typename... Args>
4      void AddClockRegion(const std::string& name, Args&&... args);
5      template <typename T>
6      void ConnectCRCInterface(const std::string& output_path, const
std::string& input_path);
7      template <typename T>
8      T* GetBlock(const std::string& path);
9
10 private:
11     virtual void Build() = 0;
12     virtual void Connect() {}
13     virtual void PostElaboration() {}
14     virtual bool IsIdle() const = 0;

```

```

15 };
16
17 class EslClockRegion : public EslComponent {
18 protected:
19     void SetPeriod(int period);
20     template <typename T, typename... Args>
21     void SetBlock(const std::string& name, Args&&... args);
22 };
23
24 class EslBlock : public EslComponent {
25 public:
26     EslBlock(const std::string& name, EslComponent* parent)
27     virtual void Clock() {}
28
29 protected:
30     template <typename T, typename... Args>
31     T* AddBlock(const std::string& name, Args&&... args);
32     template <typename T>
33     void ConnectInterface(const std::string& output_path, const std::string&
input_path);
34     void Schedule(unsigned int delay, ScheduledFn fn);
35
36 private:
37     virtual void Build() {}
38     virtual void Connect() {}
39     virtual void UpdateStatistics() {}
40 };

```

model整体结构输出文件为 `esloutput/hierarchy.log`

block间、clock region间、engine间存在可配置的并行仿真调度。

2. Config

配置文件 `eslmodel.toml`

代码块

```

1 // block class定义中声明config
2 DECLARE_CONFIG(int, pipe_num, "parameters.pipe_num", 0);

```

代码块

```

1 [output]
2 heart_beat_interval = 10000
3 output_on = true

```

```

4  output_dir_path = "esloutput"
5  statistics_on = true
6  counter_sampling_on = true
7  counter_sampling_interval = 10
8  vec_on = true
9
10 [scheduling]
11 threading = false
12 thread_num = 3
13 greedy_threading = true
14 work_balancing_interval = 100
15 sync_channel = true
16
17 [parameters]
18 pipe_num = 3
19
20 [parameters.clock_region1]
21 frequency = 3_000_000_000
22
23 [parameters.clock_region2]
24 frequency = 1_000_000_000
25
26 [parameters.clock_region3]
27 frequency = 1_000_000_000
28
29 [parameters.block1]
30 fifo_size = 2
31 pipeline_latency = 2
32 latency_pipeline_latency = 2
33

```

3. 模块开发

3.1 Block Base & Interface

ESL block 初始流程

代码块

```

1  // 同时钟域block interface连接, 子block之间的连接在父block完成
2  void Block2::Connect() {
3      ConnectInterface<Packet1>("source.output", "block1.input");
4  }
5
6  // 跨时钟域block interface连接在engine完成
7  void TestEngine::Connect() {

```

```

8     for (int pipe_id = 0; pipe_id < m_pipe_num; ++pipe_id) {
9         ConnectCRCInterface<Packet1>
            (GetComponentPath("clock_region1.block2.block1.output[]", pipe_id),
            GetComponentPath("clock_region2.sink.input[]", pipe_id));
10    }
11 }

```

3.2 模块代码结构

代码块

```

1  #include "block1.h"
2
3  void Block1::Build() {
4  }
5
6  void Block1::Clock() {
7      ProcessStage4();
8      ProcessStage3();
9      ProcessStage2();
10     ProcessStage1();
11 }
12
13 void Block1::ProcessStage1() {
14 }
15
16 void Block1::ProcessStage2() {
17 }
18
19 void Block1::ProcessStage3() {
20 }
21
22 void Block1::ProcessStage4() {
23 }

```

反压和latency的模拟需要注意pipeline上下游的调用顺序。

3.3 通用硬件结构

3.3.1 Port

input/output

代码块

```

1 // block class定义中声明input
2 DECLARE_INPUT(Packet1, test_input0);
3 DECLARE_INPUT_VEC(Packet1, 2, test_input1);
4 DECLARE_INPUT_VEC2(Packet1, 2, 2, test_input2);
5
6 // block class定义中声明output
7 DECLARE_OUTPUT(Packet1, test_output0);
8 DECLARE_OUTPUT_VEC(Packet1, 2, test_output1);
9 DECLARE_OUTPUT_VEC2(Packet1, 2, 2, test_output2);
10
11 // 接口
12 template <typename T>
13 class EsInput {
14 public:
15     bool CanGet() const;
16     EsIPacketPtr<T> Front();
17     EsIPacketPtr<T> Get();
18 };
19
20 template <typename T>
21 class EsOutput {
22 public:
23     bool CanSend() const;
24     void Send(const EsIPacketPtr<T>& pkt);
25 };

```

若Block base class为上文生成得到，则input/output已在base class中定义。

线程安全约束：在simulation阶段（elaboration阶段之后）block间传递信息必须通过port，block间不能共享非只读数据（比如直接通过指针修改其他block数据）；通过port传输的packet ptr为类std::shared_ptr语义，上下游block不能同时对packet内容读写。

3.3.2 EsFifo

fifo/buffer，内置性能数据收集。

代码块

```

1 // block class定义中声明fifo
2 DECLARE_FIFO(Packet1, test_fifo0, INF);
3 DECLARE_FIFO_VEC(Packet1, 2, test_fifo1, 4);
4 DECLARE_FIFO_VEC2(Packet1, 2, 2, test_fifo2, INF);
5
6 // 接口
7 template <typename T>
8 class EsFifo {
9 public:

```

```

10     size_t GetCapacity() const;
11     bool Empty() const;
12     bool Full() const;
13     void Push(const T& t);
14     T& Front();
15     const T& Front() const;
16     void Pop();
17     T& operator[](size_t index);
18     const T& operator[](size_t index) const;
19     iterator begin();
20     const_iterator begin();
21     iterator end();
22     const_iterator end();
23 };

```

3.3.3 EslPipeline/EslLatencyPipeline

pipeline，模拟多级流水线，提供延时/打拍/反压功能，内置性能数据收集。

代码块

```

1  // block class定义中声明pipeline
2  DECLARE_PIPELINE(Packet1, test_pipeline0, 2);
3  DECLARE_PIPELINE_VEC(Packet1, 2, test_pipeline1, 2);
4  DECLARE_PIPELINE_VEC2(Packet1, 2, 2, test_pipeline2, 2);
5
6  DECLARE_LATENCY_PIPELINE(Packet1, test_latency_pipeline0, 2);
7  DECLARE_LATENCY_PIPELINE_VEC(Packet1, 2, test_latency_pipeline1, 2);
8  DECLARE_LATENCY_PIPELINE_VEC2(Packet1, 2, 2, test_latency_pipeline2, 2);
9
10 // 接口
11 template <typename T>
12 class EslPipeline {
13 public:
14     bool FrontEmpty() const;
15     EslPacketPtr<T>& Front();
16     const EslPacketPtr<T>& Front() const;
17     void PopFront();
18     bool BackFull() const;
19     void PushBack(const EslPacketPtr<T>& pkt);
20     void Clock();
21     EslPacketPtr<T>& operator[](size_t index);
22     const EslPacketPtr<T>& operator[](size_t index) const;
23     iterator begin();
24     const_iterator begin() const;
25     iterator end();
26     const_iterator end() const;

```

```

27 };
28
29 template <typename T>
30 class EsLLatencyPipeline {
31 public:
32     bool FrontEmpty() const;
33     EsLPacketPtr<T>& Front();
34     const EsLPacketPtr<T>& Front() const;
35     void PopFront();
36     bool BackFull() const;
37     void PushBack(const EsLPacketPtr<T>& pkt);
38     EsLPacketPtr<T>& operator[](size_t index);
39     const EsLPacketPtr<T>& operator[](size_t index) const;
40     iterator begin();
41     const_iterator begin() const;
42     iterator end();
43     const_iterator end() const;
44 };

```

3.4 性能数据采集

3.4.1 Counter

代码块

```

1 // block class定义中声明counter
2 DECLARE_COUNTER(test_cnt0);
3 DECLARE_COUNTER_VEC(2, test_cnt1);
4 DECLARE_COUNTER_VEC2(2, 2, test_cnt2);
5
6 // 接口
7 class EsLCounter {
8 public:
9     operator uint64_t&();
10 };

```

输出文件:

esloutput/counters.log

esloutput/countersamples.log

3.4.2 DurationEvent

代码块

```

1 // block class定义中声明duration event
2 DECLARE_DURATION_EVENT(test_duration_event0);
3 DECLARE_DURATION_EVENT_VEC(2, test_duration_event1);
4 DECLARE_DURATION_EVENT_VEC2(2, 2, test_duration_event2);
5
6 // 接口
7 class EslDurationEvent {
8 public:
9     void Begin(const std::string& id);
10    void End(const std::string& id);
11 };

```

输出文件:

```
esloutput/durationevents.log
```

3.4.3 InstantEvent

代码块

```

1 // block class定义中声明instant event
2 DECLARE_INSTANT_EVENT(test_instant_event0);
3 DECLARE_INSTANT_EVENT_VEC(2, test_instant_event1);
4 DECLARE_INSTANT_EVENT_VEC2(2, 2, test_instant_event2);
5
6 // 接口
7 class EslInstantEvent {
8 public:
9     void Record(const std::string& msg);
10 };

```

输出文件:

```
esloutput/instantevents.log
```

3.5 Log

代码块

```

1 // block class定义中声明log
2 DECLARE_LOG(test_log0);
3 DECLARE_LOG_VEC(2, test_log1);
4 DECLARE_LOG_VEC2(2, 2, test_log2);
5
6 // 接口
7 class EslLog {

```

```

8 public:
9     bool IsValid() const;
10    EsLog& operator <<(const std::string& msg);
11 };
12
13 bool EsBlock::IsVerbose(int verbosity_level) const;
14 void EsBlock::Log(EsLog& log, int verbosity_level, const std::string& msg);

```

3.6 Misc

代码块

```

1 // 获得当前block的name/global path
2 const std::string& EsBlock::GetName() const;
3 const std::string& EsBlock::GetPath() const;
4
5 // 获得当前仿真时间和当前时钟域的cycle数
6 const SimTime* EsBlock::GetSimTime() const;
7
8 /*
9 struct SimTime {
10     uint64_t time; // in ns
11     uint64_t cycle;
12 };
13 */
14
15 // 调度仿真行为到未来cycle
16 void EsBlock::Schedule(unsigned int delay_cycle, unsigned int phase,
    ScheduleMode mode, ScheduledEvent::Fn fn)

```

4. 外部代码模块集成

4.1 Cycle Based

4.2 Event Based

5. Vector Dump

Block/Module接口数据Dump, Block/Subsystem/System level verification。

6. 性能工具

6.1.1 perf data post processing (Python)

hierarchical data/metrics

6.1.2 perf report gui (Web)

Nsight System/Compute like

table/chart/timeline/other

🔗 同步块 [该内容不支持导出查看]

7. Example

https://gitlabsoft.siorigin.com/arch/sipu-esl/models/esl_engine/-/tree/main/tests/test_engine

